Philipps Universität Marburg



Database Systems
Normalization

Thorsten Papenbrock

Normalization

1. Requirement Analysis
   - Informal description
2. Conceptional Design
   - Entity-Relationship Diagram
3. Distribution Design
   - Partitioning
4. Logical Design
   - Relation schemas
5. Data Definition
   - SQL DDL
6. Physical Design
   - Parameters and Indices
7. Implementation and Maintenance
   - Installationen, Backups, etc.



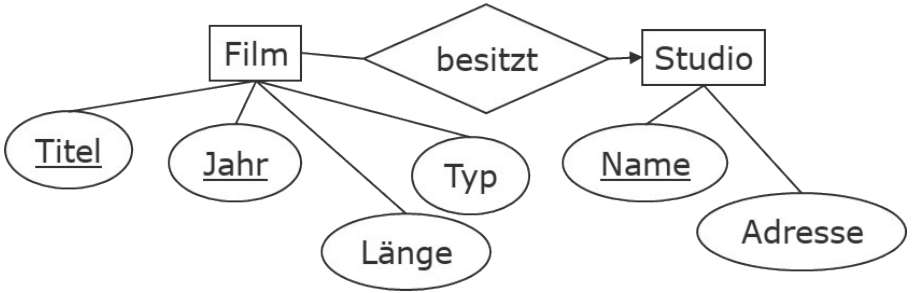
Normalization

- **Functional Dependencies**
- Closure Algorithm
- Boyce-Codd Normal Form
- Normal Forms



Functional Dependencies
Design Process: Existing Steps so far

Conceptional Design



Logical Design (Translation)

Film

Titel	Jahr	Länge	Typ
Basic Instinct	1992	127	Farbe
Total Recall	1990	113	Farbe
Dead Man	1995	121	s/w

Besitzt

Titel	Jahr	studioName
Basic Instinct	1992	Fox
Total Recall	1990	Disney
Dead Man	1995	Paramount

Studio

Name	Adresse
Fox	Hollywood
Disney	Hollywood
Paramount	Los Angeles

Logical Design (Merging)

Film

Titel	Jahr	Länge	Typ	studioName
Basic Instinct	1992	127	Farbe	Fox
Total Recall	1990	113	Farbe	Disney
Dead Man	1995	121	s/w	Paramount

Studio

Name	Adresse
Fox	Hollywood
Disney	Hollywood
Paramount	Los Angeles

Further improvements are possible!

Functional Dependencies
Motivation

- Improvements (= Refinement of the logical design)
  - Improve attributes
    (z.B. break down structures make them explicit)
  - Resolve constraints
    (z.B. split schemas to make constraints explicit)
    - Functional Dependencies **!**
    - Multivalued Dependencies

- Goal: Avoid redundancies
- Approach: Split relation schemas
  - Without losing semantic information
  - Without losing the possibility to reconstruct the relations
- Systematics: Normalization

Functional Dependencies
Definition

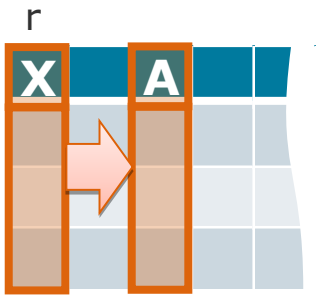
German: Funktionale Abhängigkeit (FA)

Informal:

- A functional dependency (FD) X → A is an expression over a relation R: Whenever there are two tuples in the values of the attribute set X, they also agree in the attribute value of A.

Formal:

- A functional dependency (FD) X → A in a relation schema R with a relational instance r and X ⊆ R and A ∈ R is a valid expression, if and only if
  $\forall t_i, t_j \in r : t_i[X] = t_j[X] \Rightarrow t_i[A] = t_j[A]$.



- Notation
  - A, B, C, … are attributes
  - …, X, Y, Z are attribute sets
  - X → A: „X functionally determines A.“
- Short form:
  - For ABC: {A, B, C}
  - For X → A, X → B, X → C:
    - X → ABC or
    - X → Y with Y = {A, B, C}

Functional Dependencies
Example

Philipps Universität Marburg

Name ☾ Weight

Type ☾ Weak

ID	Name	Evolution	Location	Sex	Weight	Size	Type	Weak	Strong	Special
25	Pikachu	Raichu	Viridian Forest	m/w	6.0	0.4	electric	ground	water	false
27	Sandshrew	Sandslash	Route 4	m/w	12.0	0.6	ground	gras	electric	false
29	Nidoran	Nidorino	Safari Zone	m	7.0	0.5	poison	ground	gras	false
32	Nidoran	Nidorina	Safari Zone	w	7.0	0.4	poison	ground	gras	false
37	Vulpix	Ninetails	Route 7	m/w	9.9	0.6	fire	water	ice	false
38	Ninetails	null	null	m/w	19.9	1.1	fire	water	ice	true
63	Abra	Kadabra	Route 24	m/w	19.5	0.9	psychic	ghost	fighting	false
64	Kadabra	Alakazam	Cerulean Cave	m/w	56.5	1.3	psychic	ghost	fighting	false
130	Gyarados	null	Fuchsia City	m/w	235.0	6.5	water	electric	fire	false
150	Mewtwo	null	Cerulean Cave	null	122.0	2.0	psychic	ghost	fighting	true

Location, Weight ☾ Type

Functional Dependencies
The Splitting/Combining Rule

Splitting Rule

- $A_1,A_2,\ldots A_n \rightarrow B_1,B_2,\ldots,B_m$
- $A_1,A_2,\ldots A_n \rightarrow B_i$ for i=1,…,m

Combining Rule

- $A_1,A_2,\ldots A_n \rightarrow B_i$ for i=1,…,m
- $A_1,A_2,\ldots A_n \rightarrow B_1,B_2,\ldots,B_m$

Examples

- Title, Year → Length
- Title, Year → Type
- Title, Year → StudioName
- Title, Year → Length, Type, StudioName

The splitting rule only on right-hand side!

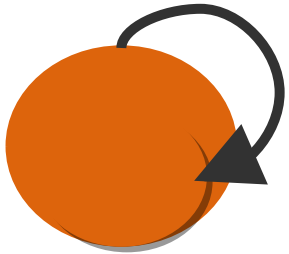
- Title, Year → Length
  - ⇏ Year → Length
  - ⇏ Title → Length

Functional Dependencies
Axioms

Axioms are derivation rules, with which further FDs are derived from the previous ones.

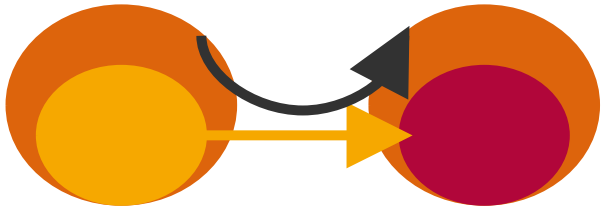
Reflexivity

- $Y \subseteq X \Rightarrow X \rightarrow Y$
- ➢ Triviality!



Augmentation

- $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
- ➢ Minimality!



Transitivity

- $X \rightarrow Y \wedge Y \rightarrow Z \Rightarrow X \rightarrow Z$
- ➢ Closure Algorithm!



The following properties are known as Armstrong Axioms:

- **Sound**
  Rules don't derive non-derivable FDs.
- **Complete**
  Rules can generate all derivable FDs.
- **Minimal**
  No rules can be canceled out.

Functional Dependencies
Axioms

Reflexivity

- $Y \subseteq X \Rightarrow X \rightarrow Y$
- ➢ Triviality!

Augmentation

- $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
- ➢ Minimality!

Transitivity

- $X \rightarrow Y \wedge Y \rightarrow Z \Rightarrow X \rightarrow Z$
- ➢ Closure Algorithm!

Commutativity does not apply!

Proof

- Given:
  - $X \rightarrow Y$
- By counterexample:

X	Y	Z
a_1	b_1	c_1
a_2	b_1	c_1
a_3	b_2	c_1
a_4	b_1	c_1

  - $Y \nrightarrow X$
- q.e.d.

Axioms

Reflexivity

- $Y \subseteq X \Rightarrow X \rightarrow Y$
- ➢ Triviality!

Augmentation

- $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
- ➢ Minimality!

Transitivity

- $X \rightarrow Y \wedge Y \rightarrow Z \Rightarrow X \rightarrow Z$
- ➢ Closure Algorithm!

Proof

- Given:
  - $X \rightarrow Y \wedge Y \rightarrow Z$
- To show:
  - Any two tuples, that agree in X, must agree in Z as well.
- Select:
  - Any two tuples, that agree in X: (x, y_1, z_1) and (x, y_2, z_2)
- Since $X \rightarrow Y \Rightarrow (x, y, z_1)$ and (x, y, z_2)
- Since $Y \rightarrow Z \Rightarrow (x, y_1, z_1)$ and (x, y_2, z_2)
- q.e.d.

Functional Dependencies
Axioms

Reflexivity

- $Y \subseteq X \Rightarrow X \rightarrow Y$
- ➢ Triviality!

Augmentation

- $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
- ➢ Minimality!

Transitivity

- $X \rightarrow Y \wedge Y \rightarrow Z \Rightarrow X \rightarrow Z$
- ➢ Closure Algorithm!

Splitting

- $X \rightarrow YZ \Rightarrow X \rightarrow Y$
- ➢ Split!

Combining

- $X \rightarrow Y \wedge X \rightarrow Z \Rightarrow X \rightarrow YZ$
- ➢ Combine!

Pseudotransitivity

- $X \rightarrow Y \wedge WY \rightarrow Z \Rightarrow WX \rightarrow Z$
- ➢ Closure Algorithm!

Derive:

$X \rightarrow Y \wedge WY \rightarrow Z$
$\Rightarrow WX \rightarrow WY \wedge WY \rightarrow Z$ (Akk.)
$\Rightarrow WX \rightarrow Z$ (Tran.)

Properties – Triviality

Trivial FD

- Attributes on the right are a subset of the ones on the left.
- Example: Title, Year → Title
- Always applies!
  - ➢ No relational instance can violate trivial FDs!

„Two tuples, that agree in a set of attributes, do always agree in each of these attributes."

Non-trivial FD

- At least an attribut on the right rechts kommt links nicht vor.
- Example: Title, Year → Year, Length

Strictly speaking, these are two FAs, so the "entirely" part is only relevant if you consider FDs to be "united".

Fully non-trivial FD

- The attributes on the left and right are disjoint.
- Example: Title, Year → Length

Furthermore, we are interested in these fully non-trivial FDs.

Formal: Trivial Dependency Rule

$A_1,A_2,\ldots A_n \rightarrow A_{i1}, \ldots, A_{ik},B_1,B_2,\ldots,B_m$ ☂ $A_1,A_2,\ldots A_n \rightarrow B_1,B_2,\ldots,B_m$

Properties – Minimality

Minimal FD

- A functional dependency $X \rightarrow A$ is minimal, iff
  $\forall B \in X : X \setminus B \rightarrow A$ is invalid.

- Non-minimal FDs can be generated (using Augmentation) from minimal FDs.
- Especially important, in order to represent a set of FDs as compact as possible and yet complete.

➢ Furthermore, we are interested only in minimal FDs.

Functional Dependencies
Use-Cases

- Schema normalization 
- Query optimization
- Data cleaning
- Schema design
- Integrity checking
- Data exploration
- Fraud detection
- Feature engineering for ML
- Data augmentation
- Data Integration
- …

Functional Dependencies
Example: Normalization

Philipps Universität Marburg

ID	Name	Size	Type	Weak	Strong	GYM	Leader	Reward
25	Pikachu	0.4	electric	ground	water	Vermillion	Lt. Surge	Thunder
26	Raichu	0.8	electric	ground	water	Vermillion	Lt. Surge	Thunder
29	Nidoran	0.5	poison	ground	gras	Viridian	Giovanni	Earth
37	Vulpix	0.6	fire	water	ice	null	null	null
38	Ninetails	1.1	fire	water	ice	null	null	null
63	Abra	0.9	psychic	ghost	fighting	null	null	null
64	Kadabra	1.3	psychic	ghost	fighting	Saffron	Sabrina	Marsh
65	Alakazam	1.5	psychic	ghost	fighting	Saffron	Sabrina	Marsh
150	Mewtwo	2.0	psychic	ghost	fighting	null	null	null

http://bulbapedia.bulbagarden.net

Functional Dependencies
Example: Normalization

Philipps Universität Marburg

Redundancy!

ID	Name	Size	Type	Weak	Strong	GYM	Leader	Reward
25	Pikachu	0.4	electric	ground	water	Vermillion	Lt. Surge	Thunder
26	Raichu	0.8	electric	ground	water	Vermillion	Lt. Surge	Thunder
29	Nidoran	0.5	poison	ground	gras	Viridian	Giovanni	Earth
37	Vulpix	0.6	fire	water	ice	null	null	null
38	Ninetails	1.1	fire	water	ice	null	null	null
63	Abra	0.9	psychic	ghost	fighting	null	null	null
64	Kadabra	1.3	psychic	ghost	fighting	Saffron	Sabrina	Marsh
65	Alakazam	1.5	psychic	ghost	fighting	Saffron	Sabrina	Marsh
150	Mewtwo	2.0	psychic	ghost	fighting	null	null	null

Type → Weak, Strong

GYM → Leader, Reward

http://bulbapedia.bulbagarden.net

Functional Dependencies
Example: Normalization



-25 Rows!

ID	Name	Size	Type	GYM
25	Pikachu	0.4	electric	Vermillion
26	Raichu	0.8	electric	Vermillion
29	Nidoran	0.5	poison	Viridian
37	Vulpix	0.6	fire	null
38	Ninetails	1.1	fire	null
63	Abra	0.9	psychic	null
64	Kadabra	1.3	psychic	Saffron
65	Alakazam	1.5	psychic	Saffron
150	Mewtwo	2.0	psychic	null

http://bulbapedia.bulbagarden.net

Type	Weak	Strong
electric	ground	water
poison	ground	gras
fire	water	ice
psychic	ghost	fighting

GYM	Leader	Reward
Vermillion	Lt. Surge	Thunder
Viridian	Giovanni	Earth
Saffron	Sabrina	Marsh

Properties – Authenticity

Definition (Recap)

- A functional dependency (FD) $X \rightarrow A$ in a relation schema R with a relational instance r and $X \subseteq R$ and $A \in R$ is a valid statement, if and only if $\forall t_i, t_j \in r : t_i[X] = t_j[X] \Rightarrow t_i[A] = t_j[A]$.

Authentic FD

- An FD might randomly be valid for an instance, however, not for another instance in the same relation schema.
  - e.g. ZipCode → CityName is almost always correct
    - However: „Oberkrämer" and „Velten" share the same zip code „16727"
- An authentic FD is an statement about the relation schema R, that must be valid for **all** relations $r \in REL(R)$ of the relation schema!

Functional Dependencies
Properties – Authenticity

Philipps Universität Marburg

Example: Relation on Films

Title	Year	Length	Type	StudioName	ActorName
Total Recall	1990	113	Color	Fox	Sharon Stone
Basic Instinct	1992	127	Color	Disney	Sharon Stone
Total Recall	1990	113	Color	Fox	Arnold
Dead Man	1995	121	BW	Paramount	Johnny Depp





- Title → Type looks valid
  - However: only for this instance by chance!
  - Counterexample: „King Kong" from „1924" is „BW"
    „King Kong" from „2005" is „Color"
- Title, Year → Type is valid on the other hand

➢ Furthermore, we always accept authentic FDs!

Functional Dependencies
Origin

Where do FDs come from?

- Declaration by the developer in the database design:
  - The developer defines a dependency as functional (e.g. Position → Salary)
  - DBMS should ensure, that the FD for relations is always valid.
- Physical/Logical existence in the real world:
  - Each recorded relation naturally satisfies the FD (e.g. BirthDate → Age)
  - Natural FDs can be found in the relations (automatically).
- Key from the database design:
  - Each key defines a set of FDs (e.g. StNr → Name, Year, …)
  - Key K for schema R defines the FDs K → R.
  - Each relation has a key (in case of a doubt, all attributes together)

FDs as Integrity Constraints

- **Declaration by the developer in the database design**
- Physical/Logical existence in the real world
- Key from the database design

- If the FD X → A is valid, then the following query may have **no results**:

```
SELECT A, COUNT(DISTINCT B)
FROM R
GROUP BY A
HAVING COUNT(DISTINCT B) > 1;
```

- If the FD X → A is ensured, then the query describes an integrity constraint, that is checked during each update.
  - e.g. during insertion of tuples (…., a, …., b,…) test whether the following query has an empty result:

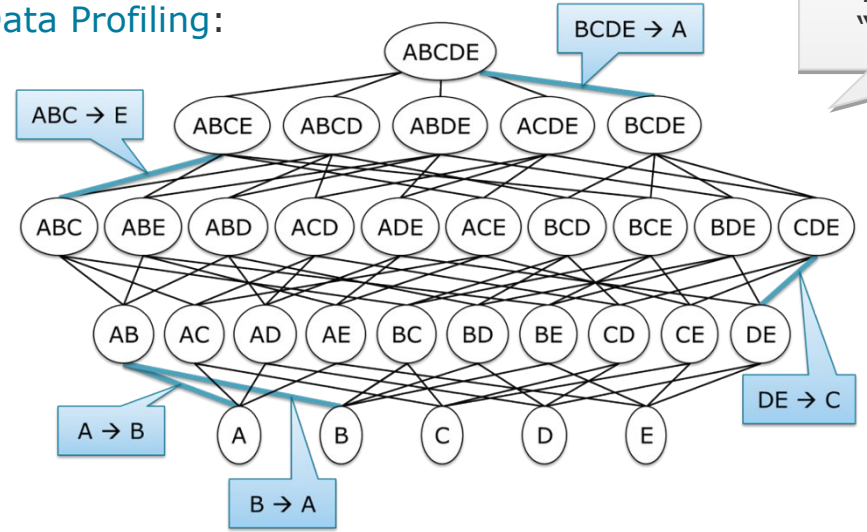
```
SELECT *
FROM R
WHERE A = a AND B <> b;
```

With an index on A logarithmic

FD Detection by Data Profiling

- Declaration by the developer in the database design
- **Physical/Logical existence in the real world**
- Key from the database design

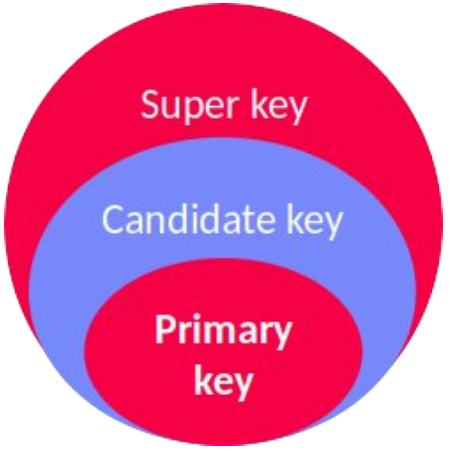
- Natural FDs are not often known (in the beginning), however, they can automatically be found using Data Profiling:
  - Search space:
    All possible FD candidates; here as Power Set Lattice
  - Validation:
    e.g. with the SQL query from before



More in the course "Data Integration".

FDs and Keys

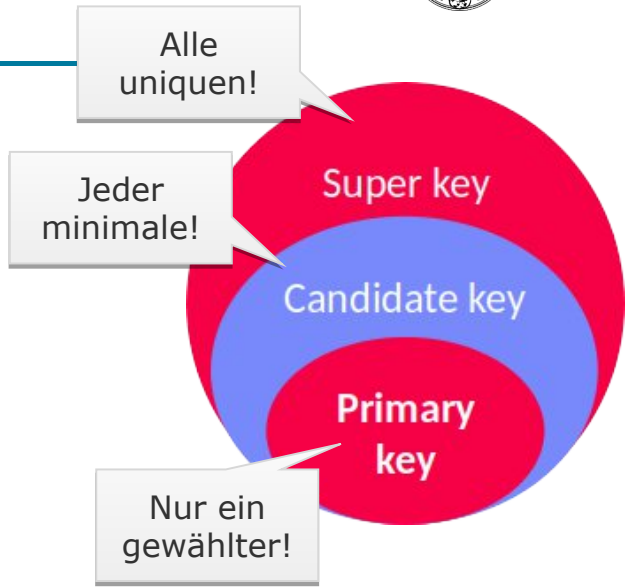
- Declaration by the developer in the database design
- Physical/Logical existence in the real world
- **Key from the database design**



- A set of one or more attributes $\{A_1, A_2, \ldots, A_n\}$ is a key of the relation R, iff the following is valid:
  1. The attributes determine all (other) attributes functionally: $\{A_1, A_2, \ldots, A_n\} \rightarrow R$
     - Recall: Since the relations are sets, there are no two completely identical tuples, so that $\{A_1, A_2, \ldots, A_n\}$ may have duplicate values, if $\{A_1, A_2, \ldots, A_n\} \rightarrow R$.
  2. No real subset functionally determines all (other) attributes: $\nexists\, X \subset \{A_1, A_2, \ldots, A_n\}: X \rightarrow R$
     - Recall: A key must therefore be minimal.

FDs and Keys – Terminology

- A relation can consist of more than one key.
  - Select one of the keys as primary key
  - Rename all the other keys to key candidates
- In practice, non-minimal keys are often relevant as well
  - Rename non-minimal keys super keys
    - {Title, Year, ActorName} is the key and super key
    - {Title, Year, Length, ActorName} is a super key



- Minimal vs. Smallest
  - Minimal Key: No attribute may be missing
    - ➢ is not exactly the smallest key!
  - The Smallest Key: Key with the fewest attributes
    - ➢ Is also a minimal key!

Functional Dependencies
FDs and Keys

- Example: Relation on Films

Title	Year	Length	Type	StudioName	ActorName
Total Recall	1990	113	Color	Fox	Sharon Stone
Basic Instinct	1992	127	Color	Disney	Sharon Stone
Total Recall	1990	113	Color	Fox	Arnold
Dead Man	1995	121	BW	Paramount	Johnny Depp

- {Title, Year, ActorName} is a key.
  - No two tuples can have the same values for Title, Year and ActorName. Otherwise, they would be completely identical!
- {Title, Year} → {Length, Type, Studioname} is a functional dependency.
- Subsets?
  - {Title, Year} ↛ ActorName
  - {Year, ActorName} ↛ Title
  - {Title, ActorName} ↛ Year

{Title, Year, ActorName} is minimal!

Functional Dependencies
FDs and Redundancy

Redundanz!

- Functional dependencies, that are not a part of a key, describe redundant data
- Example: Pokemon Relation
  - Type properties
  - GYM details
  - ➢ Both are saved multiple times, although they cannot be differentiated on the left-hand side.

ID	Name	Size	Type	Weak	Strong	GYM	Leader	Reward
25	Pikachu	0.4	electric	ground	water	Vermillion	Lt. Surge	Thunder
26	Raichu	0.8	electric	ground	water	Vermillion	Lt. Surge	Thunder
29	Nidoran	0.5	poison	ground	gras	Viridian	Giovanni	Earth
37	Vulpix	0.6	fire	water	ice	null	null	null
38	Ninetails	1.1	fire	water	ice	null	null	null
63	Abra	0.9	psychic	ghost	fighting	null	null	null
64	Kadabra	1.3	psychic	ghost	fighting	Saffron	Sabrina	Marsh
65	Alakazam	1.5	psychic	ghost	fighting	Saffron	Sabrina	Marsh
150	Mewtwo	2.0	psychic	ghost	fighting	null	null	null

Type → Weak, Strong **GYM → Leader, Reward**

- Consequences:
  - Increased Storage Usage
    (e.g. multiple Speicherung of Leader „Lt. Surge“)
  - Increased Costs during Updates
    (e.g. new leader in „Vermillion“ affects 2 tuples)
  - Increased Susceptibility for Inconsistencies and Anomalies
    (e.g. if a leader isn’t update all tuples consistently)

Update Anomaly
Redundant values on the right-hand side of an FD are updated not consistently whenever necessary!

Delete Anomaly
With the deletion of the last Pokemon of a type or a gym, the information of Type and GYM are also lost!

Insert Anomaly
New Types and GYMs can not be updated if there is no Pokemons in these columns!

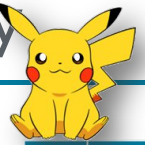
- Functional dependencies, that are not a part of a key, describe redundant data
- Example: Pokemon Relation
  - Type properties
  - GYM details
  - ➢ Both are saved multiple times, although they cannot be differentiated on the left-hand side.

ID	Name	Size	Type	Weak	Strong	GYM	Leader	Reward
25	Pikachu	0.4	electric	ground	water	Vermillion	Lt. Surge	Thunder
26	Raichu	0.8	electric	ground	water	Vermillion	Lt. Surge	Thunder
29	Nidoran	0.5	poison	ground	gras	Viridian	Giovanni	Earth
37	Vulpix	0.6	fire	water	ice	null	null	null
38	Ninetails	1.1	fire	water	ice	null	null	null
63	Abra	0.9	psychic	ghost	fighting	null	null	null
64	Kadabra	1.3	psychic	ghost	fighting	Saffron	Sabrina	Marsh
65	Alakazam	1.5	psychic	ghost	fighting	Saffron	Sabrina	Marsh
150	Mewtwo	2.0	psychic	ghost	fighting	null	null	null

Type → Weak, Strong **GYM → Leader, Reward**

- Consequences:
  - Increased Storage Usage
    (e.g. multiple Speicherung of Leader „Lt. Surge“)
  - Increased Costs during Updates
    (e.g. new leader in „Vermillion“ affects 2 tuples)
  - Increased Susceptibility for Inconsistencies and Anomalies
    (e.g. if a leader isn’t update all tuples consistently)

Functional Dependencies
FDs and Redundancy



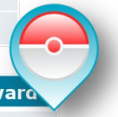
- Functional dependencies, that are not a part of a key, describe redundant data

- The goal of the database design:
  Normalization

ID	Name	Size	Type	GYM
25	Pikachu	0.4	electric	Vermillion
26	Raichu	0.8	electric	Vermillion
29	Nidoran	0.5	poison	Viridian
37	Vulpix	0.6	fire	null
38	Ninetails	1.1	fire	null
63	Abra	0.9	psychic	null
64	Kadabra	1.3	psychic	Saffron
65	Alakazam	1.5	psychic	Saffron
150	Mewtwo	2.0	psychic	null



Type	Weak	Strong
electric	ground	water
poison	ground	gras
fire	water	ice
psychic	ghost	fighting



GYM	Leader	Reward
Vermillion	Lt. Surge	Thunder
Viridian	Giovanni	Earth
Saffron	Sabrina	Marsh

- Keys …
  - avoid functional redundancies, since the left-hand side is duplicate-free.
  - As integrity constraints are easier for DBMS to ensure.
  - Make the functional relationships explicit.

- Decomposition
  - Converts FDs to the keys of the new relation through relation splits.
  - Keeps all semantic information and is reversible.

But before that a little more theory…

Normalization

- Functional Dependencies
- **Closure Algorithm**
- Boyce-Codd Normal Form
- Normal Forms



Closure Algorithm
FD Sets

Equivalence

- Two sets S and T of FDs are called equivalent, iff the sets of the valid relational instances in S the same as in T: $S^+ = T^+$
  - We can derive various FDs for S and T using the axioms, even though their "bases" form the same set of FDs.
    - ➢ They are perhaps unequal but equivalent

Conclusion

- A set S of FDs follow from a set T of FDs, iff each set in T is also a valid relational instance in S.
  - The set S can be converted to its FDs in T with the help of axioms.
    - ➢ This results in derivation paths for FA sets!

Closure Algorithm
Basis

Basis

- A set of FDs, from which all other FDs can be derived

Minimal Basis

- A basis, of which no subset is a basis

Example: R(A, B, C) and each attribute functionally determines the other two

- Valid (minimal) FDs: A→B, A→C, B→A, B→C, C→A, C→B
- Derived FDs: AB→C, AC→B, BC→A
- Short forms: A→BC, B→AC, C→AB
- Trivial FDs: A→A, B→B, C→C
- Nontrivial FDs: AB→BC, AC→BC, …
- Minimal Basis: {A→B, B→A, B→C, C→B}
- Minimal Basis: {A→B, B→C, C→A}

Closure Algorithm
Closure

- Given a set of attributes $\{A_1,A_2,\ldots,A_k\}$ and a set F of FDs.

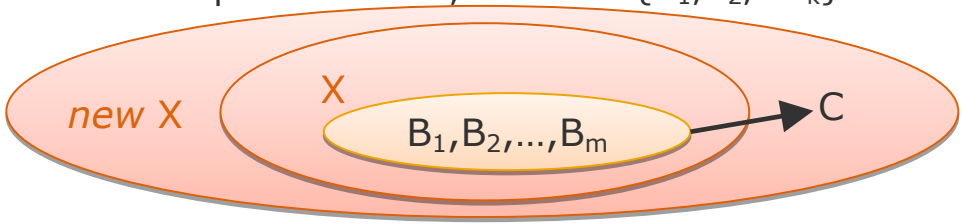
- The closure $\{A_1,A_2,\ldots,A_k\}^+$ of the set of attributes $\{A_1,A_2,\ldots,A_k\}$ of the set F of FDs is the maximal set Y of all attributes, such that for each valid relation in F is also valid $A_1,A_2,\ldots,A_k \rightarrow Y$.
  - The set of $\{A_1,A_2,\ldots,A_k\}$ with F functionally derivable attributes
  - i.e. $A_1,A_2,\ldots A_k \rightarrow Y$ applies from the FAs in F.

- Notation:
  - Closure of $\{A_1,A_2,\ldots,A_k\}$: $\{A_1,A_2,\ldots,A_k\}^+$

- It applies $A_i \in \{A_1,A_2,\ldots,A_k\}^+$ for i=1,…,k
  - Trivially, because $A_1,A_2,\ldots,A_i,\ldots,A_k \rightarrow A_i$

Closure Algorithm
Closure Algorithm

1. Initialize X as X = $\{A_1,A_2,\ldots A_k\}$.
   - X is the set of attributes that will eventually be the closure.
2. Search an FD $B_1,B_2,\ldots,B_m \rightarrow C$ with $B_1,B_2,\ldots,B_m \in X$ but $C \notin X$.
3. Add C to X.
4. Repeat steps 2 + 3 until no more new FD can be found.
   - Termination:
     - If S is the set of all FDs and R the set of all attributes, then there can be only min(|S|,|R|-|X|) many extensions at most.
5. Return X.
   - X is the computed closure, also X = $\{A_1,A_2,\ldots A_k\}^+$.



A^+ Algorithm

```
Closure(F, A)
  X = A;
  WHILE (X changed)
    FOREACH (B → C ∈ F) DO
      IF (B ∈ X)
        X = X ∪ C;
RETURN X;
```

Closure Algorithm
Example

- Relation with attributes A, B, C, D, E, F
- Given: A set with four FDs
  1. AB → C
  2. BC → AD
  3. D → E
  4. CF → B
- Search: Closure of {A, B}, also $\{A, B\}^+$
  - FD 1: X = {A, B, C}
  - FD 2: X = {A, B, C, D}
  - FD 3: X = {A, B, C, D, E} (= $\{A, B\}^+$)

Closure Algorithm
Proof

- Correctness: It generates no invalid FDs.
  - Induction on number of operations
  - Transitivity i.e. argumentation for the tuples.

- Completeness: It generates all the valid FDs.
  - Assumption of a counterexample, i.e., there is a FD $X \rightarrow Y$, that is not found.
  - Construction of an instance, that is valid for FDs, but not for $X \rightarrow Y$.

Closure Algorithm
Membership-Problem

- Question:
  Can a specific FD X → Y be derived from the given FD sets?
- Approach:
  Compute closure of X and test if Y is included in it.

- Example:
  - Given: AB → C and BC → AD and D → E and CF → B
  - Question: Can AB → D be derived?
    - $\{AB\}^+ = \{A, B, C, D, E\}$
    - D ∈ {A, B, C, D, E}, so YES!
  - Question: Can D → A be derived?
    - $\{D\}^+ = \{D, E\}$
    - A ∉ {D, E}, so NO!

Closure Algorithm
FDs after Projections

Philipps Universität Marburg

- Motivation: Normalization breaks down a relation R with the original FD set F into many small relations; only a part of F is valid in each result relation.
- Question: Let S ⊂ R be the result of the projection of few attributes from R.
  Which FDs are still valid for S?
  - ➢ All FDs, that (1) apply from F and (2) use only attributes from S.
- Example: R(A, B, C, D)
  - FDs: {A → B, B → C, C → D}
  - Projection of B: S(A, C, D)
- Algorithm: Compute the closure of each subset!
  - Systematics: (1) the length of 1, (2) the length of 2, … (|S| - 1) the length of |S| - 1
    - Trick 1: Empty set and the set of all attributes must not be tested.
    - Trick 2: If the closure of X contains all attributes,
      the supersets of X must not be checked anymore.
  - Derivation: Derive all valid, nontrivial FDs from the closures: $X \rightarrow \{X\}^+$ / (R/S) / X

Closure Algorithm
FDs after Projections

- Example: R(A, B, C, D)
  - FDs: {A → B, B → C, C → D}
  - Projection of B: S(A, C, D)
- Algorithm: Compute the closure of each subset!
  - $\{A\}^+ = \{A, B, C, D\}$
    - A → C and A → D
    - A → B is also true, but B is not in S.
    - $\{A\}^+$ already consists of all attributes of S, therefore generate no further supersets.
  - $\{C\}^+ = \{C, D\}$
    - C → D
  - $\{D\}^+ = \{D\}$
  - $\{C,D\}^+ = \{C,D\}$
  - Result: A → C, A → D and C → D

We take only nontrivial FDs into account for the calculations!

Minimal Basis: {A → C, C → D}

Normalization

- Functional Dependencies
- Closure Algorithm
- **Boyce-Codd Normal Form**
- Normal Forms



Boyce-Codd Normal Form
Redundancy

Redundanz!

- Functional dependencies, that are not a part of a key, describe redundant data
  - Weak and Strong of each Types shown several times
  - Leader and Reward of each Arena shown several times

ID	Name	Size	Type	Weak	Strong	GYM	Leader	Reward
25	Pikachu	0.4	electric	ground	water	Vermillion	Lt. Surge	Thunder
26	Raichu	0.8	electric	ground	water	Vermillion	Lt. Surge	Thunder
29	Nidoran	0.5	poison	ground	gras	Viridian	Giovanni	Earth
37	Vulpix	0.6	fire	water	ice	null	null	null
38	Ninetails	1.1	fire	water	ice	null	null	null
63	Abra	0.9	psychic	ghost	fighting	null	null	null
64	Kadabra	1.3	psychic	ghost	fighting	Saffron	Sabrina	Marsh
65	Alakazam	1.5	psychic	ghost	fighting	Saffron	Sabrina	Marsh
150	Mewtwo	2.0	psychic	ghost	fighting	null	null	null

Type → Weak, Strong **GYM → Leader, Reward**

- Update Anomalies
  - The change of an arena leader requires a consistent update of many tuples.
- Insert Anomalies
  - Insertion of a new Pokemon requires the check of Type-Weak combination.
- Delete Anomalies
  - After deletion of the last fire Pokemon, Fire-Ice strength will get lost.

Boyce-Codd Normal Form
Decomposition

- Goal: Elimination of anomalies during decomposition of the affected relation

- Decomposition
  1. Splitting the attributes into two relations
  2. Generating the tuples in the new two relations

- $R(A_1,A_2,\ldots,A_k)$ can be decomposed into $S(B_1,B_2,\ldots,B_m)$ and $T(C_1,C_2,\ldots,C_l)$, iff
  1. $\{A_1,A_2,\ldots,A_k\} = \{B_1,B_2,\ldots,B_m\} \cup \{C_1,C_2,\ldots,C_l\}$
  2. Tuples in S are the projection of all tuples in R on $\{B_1,B_2,\ldots,B_m\}$
  3. Tuples in T are the projection of all tuples in R on $\{C_1,C_2,\ldots,C_l\}$
     - Projections remove duplicates
       - Redundancy is reduced.

Schemas can (usually should!) overlap!

Boyce-Codd Normal Form
Lossless Decomposition

- Goal: Elimination of the anomalies by decomposing the affected relations, in a way that ensures the complete restorability of the original relation

- Lossless Decomposition
  - Let R be a relation schema and F_R the set of FDs for R.
    A splitting of R in schemas S and T has **no** information lost, iff one of the following conditions applies:
    a) $(S \cap T) \rightarrow S \in F_R^+$
    b) $(S \cap T) \rightarrow T \in F_R^+$
       - S or T results in the attributes of an FD for R!
       - S and T have the attributes of the left-hand side of the FD!
  - Because of that, the relation R is restorable from the relations S ($= \pi_S(R)$) and T ($= \pi_T(R)$) using the join operation $\bowtie$:
    - $R = S \bowtie_{S \cap T} T = \pi_S(R) \bowtie_{S \cap T} \pi_T(R)$

Boyce-Codd Normal Form
Example

- Pokemon:

ID	Name	Size	Type	Weak	Strong	GYM	Leader	Reward
25	Pikachu	0.4	electric	ground	water	Vermillion	Lt. Surge	Thunder
26	Raichu	0.8	electric	ground	water	Vermillion	Lt. Surge	Thunder
29	Nidoran	0.5	poison	ground	gras	Viridian	Giovanni	Earth
37	Vulpix	0.6	fire	water	ice	null	null	null
38	Ninetails	1.1	fire	water	ice	null	null	null
63	Abra	0.9	psychic	ghost	fighting	null	null	null
64	Kadabra	1.3	psychic	ghost	fighting	Saffron	Sabrina	Marsh
65	Alakazam	1.5	psychic	ghost	fighting	Saffron	Sabrina	Marsh
150	Mewtwo	2.0	psychic	ghost	fighting	null	null	null

- FDs:
  - Type → Weak, Strong
  - GYM → Leader, Reward
- Decomposition:
  - Pokemon(ID, Name, Size, Type, $\overline{\text{Weak, Strong}}$, GYM, $\overline{\text{Leader, Reward}}$)
  - Types(Type, Weak, Strong)
  - GYMs(GYM, Leader, Reward)

Boyce-Codd Normal Form
Example

- Pokemon:

ID	Name	Size	Type	GYM
25	Pikachu	0.4	electric	Vermillion
26	Raichu	0.8	electric	Vermillion
29	Nidoran	0.5	poison	Viridian
37	Vulpix	0.6	fire	null
38	Ninetails	1.1	fire	null
63	Abra	0.9	psychic	null
64	Kadabra	1.3	psychic	Saffron
65	Alakazam	1.5	psychic	Saffron
150	Mewtwo	2.0	psychic	null

Redundancy?

Types

Type	Weak	Strong
electric	ground	water
poison	ground	gras
fire	water	ice
psychic	ghost	fighting

Redundancy?

GYMs

GYM	Leader	Reward
Vermillion	Lt. Surge	Thunder
Viridian	Giovanni	Earth
Saffron	Sabrina	Marsh

- FDs:
  - Type → Weak, Strong
  - GYM → Leader, Reward
- Decomposition:
  - Pokemon(ID, Name, Size, Type, GYM)
  - Types(Type, Weak, Strong)
  - GYMs(GYM, Leader, Reward)
- Anomalies solved:
  - Update Anomaly
  - Insert Anomaly
  - Delete Anomaly

Byce-Codd Normal Form (BCNF)

- Definition:
  - A relation R is in BCNF if and only if for each nontrivial FD $X \rightarrow A$ with $X \subseteq R$ and $A \in R$ the attributes X are any super key for R.
    - The left-hand side of each valid, nontrivial FD is a super key.
    - The left-hand side of each valid, nontrivial FD contains a key.
- Reminder:
  - Super key: Keys or super sets of a key, also $X \rightarrow R$.
  - Nontrivial: At least an attribute on the right doesn't exist on the left, also $A \notin X$.
- Example:
  - Pokemon(ID, Name, Size, Type, Weak, Strong, GYM, Leader, Reward)
  - Key: {ID}
  - FDs: Type → Weak, Strong and GYM → Leader, Reward
    - Since both $\{ID\}^+ \neq \{Type\}$ and $\{ID\}^+ \neq \{GYM\}$, BCNF is violated here!

Boyce-Codd Normal Form
BCNF Algorithm

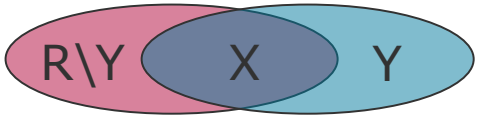
Philipps Universität Marburg

Given: Relation schema R and a set of FDs F

1. Determine keys
   - Use the closure algorithm in order to find all keys of R.
   - Select a key as primary key.
2. Expand FDs
   - Use the closure algorithm to maximally expand all FDs in F.
3. Decompose
   - Test all FDs in order to find a BCNF violating, expanding FD $X \rightarrow Y$.
   - Split the relation R of the violated FD in $R \setminus Y$ and $X \cup Y$.
     - If necessary, redetermine the primary key for $R \setminus Y$, if separated.
     - Primary key for $X \cup Y$ is X.
     - Valid FDs for $R \setminus Y$ are all $W \rightarrow V'$ with $W \rightarrow V \in F$, $V' \subseteq V$, $W \subseteq R \setminus Y$ and $V' \subseteq R \setminus Y$.
     - Valid FDs for $X \cup Y$ are all $W \rightarrow V'$ with $W \rightarrow V \in F$, $V' \subseteq V$, $W \subseteq X \cup Y$ and $V' \subseteq X \cup Y$.
   - Repeat 3 for each subrelation, until there are no violating FDs anymore.

Even if there is a primary key, we need **all** keys!

Therefore each splitting splits off maximal many attributes!



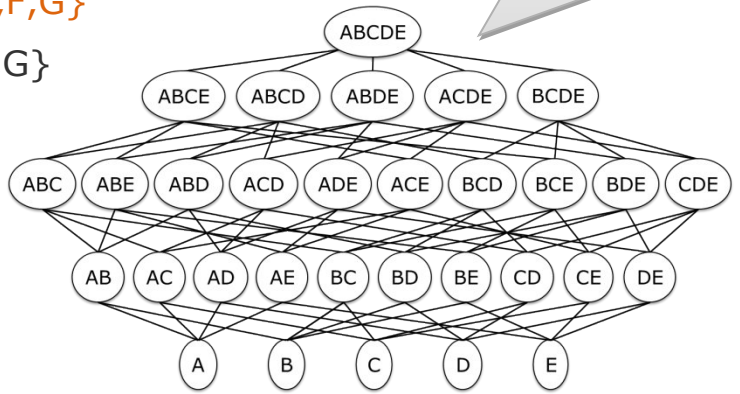
Boyce-Codd Normal Form
BCNF Algorithm – Example

- Relation: R(A,B,C,D,E,F,G)
- FDs:
  - A,B → C,D
  - C → D,E
  - D,E → F,G

1. Determine keys:
   - $\{A\}^+ = \{A\}$
   - $\{B\}^+ = \{B\}$
   - $\{C\}^+ = \{C,D,E,F,G\}$
   - $\{D\}^+ = \{D\}$
   - $\{E\}^+ = \{E\}$
   - $\{F\}^+ = \{F\}$
   - $\{G\}^+ = \{G\}$

- $\{A,B\}^+ = \{A,B,C,D,E,F,G\}$
- $\{A,C\}^+ = \{A,C,D,E,F,G\}$
- $\{A,D\}^+ = \{A,D\}$
- $\{A,E\}^+ = \{A,E\}$
- …

Exponentially many, therefore it's better to draw logical conclusions!



BCNF Algorithm – Example

- Relation: R(A,B,C,D,E,F,G)
- FDs:
  - A,B → C,D
  - C → D,E
  - D,E → F,G

1. Determine keys:
   - A and B are never on right of an FD
     - Each key must contain A and B!
   - Therefore test:
     - $\{A,B\}^+ = \{A,B,C,D,E,F,G\}$
   - Since {A,B} is already a key, test no further super sets.
   - Since no combinations are more to generate, {A,B} is single key of the relation R.

BCNF Algorithm – Example

- Relation: R(<u>A,B</u>,C,D,E,F,G)
- FDs:
  - A,B → C,D
  - C → D,E
  - D,E → F,G
- Key:
  - {A,B}

2. Expand FDs:
   - $\{A,B\}^+ = \{A,B,C,D,E,F,G\}$ A,B → C,D,E,F,G
   - $\{C\}^+ = \{C,D,E,F,G\}$ C → D,E,F,G
   - $\{D,E\}^+ = \{D,E,F,G\}$ D,E → F,G

Boyce-Codd Normal Form
BCNF Algorithm – Example

- Relation: R(A,B,C,D,E,F,G)
- FAs:
  - A,B → C,D
  - C → D,E
  - D,E → F,G
- Key:
  - {A,B}
- Expanded FDs:
  - A,B → C,D,E,F,G
  - C → D,E,F,G
  - D,E → F,G

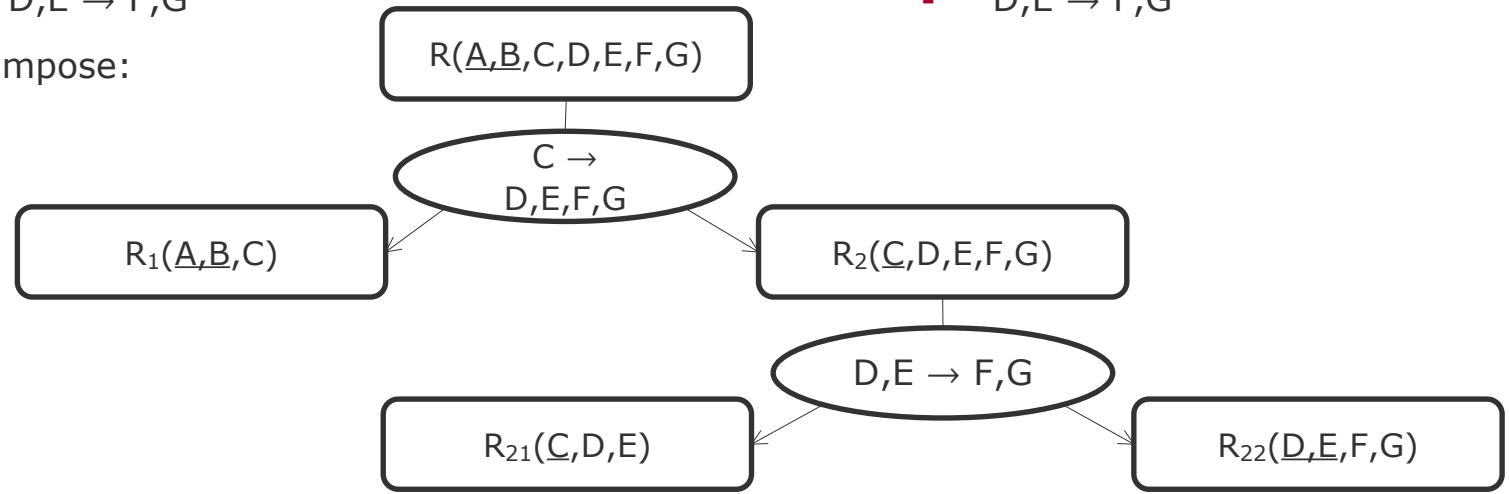
Recursive Splitting

3. Decompose:
   - BCNF violating, expanding FDs:
     - C → D,E,F,G
     - D,E → F,G
   - Select C → D,E,F,G and split:
     - R_1(A,B,C) and R_2(C,D,E,F,G)
     - Violations in R_1: None
     - Violations in R_2: D,E → F,G
   - BCNF violating, expanding FDs:
     - D,E → F,G
   - Select D,E → F,G and split:
     - R_{21}(C,D,E) and R_{22}(D,E,F,G)
     - Violations in R_{21}: None
     - Violations in R_{22}: None
   - Result: R_1(A,B,C) R_{21}(C,D,E) R_{22}(D,E,F,G)

Boyce-Codd Normal Form
BCNF Algorithm – Example

- Relation: R(A,B,C,D,E,F,G)
- FDs:
  - A,B → C,D
  - C → D,E
  - D,E → F,G
- Key:
  - {A,B}
- Expanding FDs:
  - A,B → C,D,E,F,G
  - C → D,E,F,G
  - D,E → F,G

3. Decompose:



Boyce-Codd Normal Form
BCNF with Instance Data

Philipps Universität Marburg

Example: Films

- Key:
  - {Title, Year, ActorName}
- FDs:
  - Title, Year → Length
  - Title, Year → Type
  - Title, Year → StudioName

1. BCNF Decomposition
   - Split relations recursively to BCNF.
2. Duplicate Removal
   - Delete completely identical records.

Title	Year	Length	Type	StudioName	ActorName
Total Recall	1990	113	Farbe	Fox	Sharon Stone
Basic Instinct	1992	127	Farbe	Disney	Sharon Stone
Total Recall	1990	113	Farbe	Fox	Arnold
Dead Man	1995	121	s/w	Paramount	Johnny Depp

BCNF Decomposition

Title	Year	Length	Typ	StudioName
Total Recall	1990	113	Farbe	Fox
Basic Instinct	1992	127	Farbe	Disney
Total Recall	1990	113	Farbe	Fox
Dead Man	1995	121		

Title	Year	ActorName
Total Recall	1990	Sharon Stone
Basic Instinct	1992	Sharon Stone
Total Recall	1990	Arnold
Dead Man	1995	Johnny Depp

Duplicate Removal

Title	Year	Length	Typ	StudioName
Total Recall	1990	113	Farbe	Fox
Basic Instinct	1992	127	Farbe	Disney
Dead Man	1995	121	s/w	Paramount

Title	Year	ActorName
Total Recall	1990	Sharon Stone
Basic Instinct	1992	Sharon Stone
Total Recall	1990	Arnold
Dead Man	1995	Johnny Depp

Foreign key

Boyce-Codd Normal Form
Recovering Information

- Requirement: BCNF decomposition must guarantee restorability
  - For R decomposed in S and T the following applies: $R = S \bowtie_{S \cap T} T$
    - (1) complete and (2) correct
- Assumed: R(<u>A</u>,B,C) with key {A} and FA B → C (= BCNF violation)
- Decomposition: R_1(<u>A</u>,B) and R_2(<u>B</u>,C)
- Let t = (a,b,c) be a tuple in R
  - becomes t_1(a,b) in R_1 and t_2(b,c) in R_2
- Recover by „Join" (Composite; ⋈):
  - The combination of tuples of two relations S and T, that agree in the values for common attributes S∩T.

1. Completeness
   - t_1(a,b) linked to t_2(b,c) becomes t(a,b,c)
2. Correctness
   - Next page

Boyce-Codd Normal Form
Recovering Information

2. Correctness
   - Let $t_1(a,b,c)$ and $t_2(d,b,e)$ be two tuples in R
   - Decomposition using projection
     - In R_1: t_1'(a,b) and t_2'(d,b)
     - In R_2: t_2''(b,c) and t_2''(b,e)
   - Join $R_1 \bowtie R_2$ results in tuples
     - v(a,b,c) w(a,b,e) x(d,b,e) y(d,b,c)
   - Are w(a,b,e) and y(d,b,c) mistakes?
   - No, because of BCNF decomposition B → C
     - c = e
       - $v = w = t_1$ and $x = y = t^2$
   - This also applies more generally to attribute sets

R

A	B	C
a	b	c
d	b	e =c

R_1

A	B
a	b
d	b

R_2

B	C
b	c
~~b~~	~~e~~

$R_1 \bowtie R_2$

A	B	C
a	b	c
~~a~~	~~b~~	~~e~~
d	b	c
~~d~~	~~b~~	~~e~~

Keys Destroyed – What now?

Example:

- Relation: R(<u>A,B</u>,C,D)
- Key: {A,B}
- FD: $D \rightarrow B$
  - ➢ Violates BCNF!
- BCNF compliant split: R_1(A,C,D) und R_2(B,<u>D</u>)
  - ➢ Key of R_1 is destroyed!

➢ Try the closure algorithm again to find an alternative key.

- We can only find: $\{A,C,D\}^+ = \{A,C,D\}$
- ➢ Therefore, the keys: R_1(<u>A,C,D</u>) and R_2(B,<u>D</u>)
  - Because $D \rightarrow B$ we don't lose any records in R_1 and can restore R as $R = R_1 \bowtie_D R_2$ including the key {A,B}.

Boyce-Codd Normal Form
Decomposition and ER Modeling

- Films(Title, Year, Length, Type, StudioName, ActorName)
  Title, Year → Length, Type, StudioName
  - Films(Title, Year, Length, Type, StudioName)
  - Casts(Title, Year, ActorName)

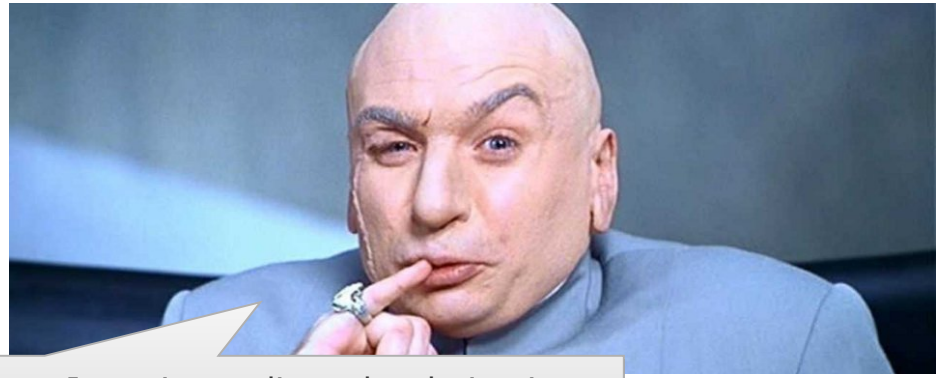
➢ BCNF violation by representation of an n:m relationship in a single relation.

- Film(Title, Year, Length, Typ, StudioName, StudioAddress)
  StudioName → StudioAddress
  - Film(Title, Year, Length Type, StudioName)
  - Studios(StudioName, StudioAdresse)

➢ BCNF violation by representation of an n:1 Beziehung in a single relation.

Boyce-Codd Normal Form
Two Attribute Split

- Any relation with only two attributes R(A,B) is in BCNF.
- Proof (Case differentiation):
  - No FDs:
    - {A,B} is the key
    - BCNF, since no violating FD
  - A → B, but not B → A
    - {A} is the key
    - BCNF, since no violating FD
  - A → B and B → A
    - {A} and {B} are keys
    - BCNF, since each FD needs only one key



Then, I can just split each relation into two-attributed relations!

Only if it works with **lossless decomposition**!

➢ Each decomposition of a relation in relations of the size of 2 provides a BCNF compliant result.

Boyce-Codd Normal Form
Two Attribute Split

- Not every decomposition in a two-attributed relation is lossless.
- Proof (Counterexample):
  - Assume R(A,B,C) without B → C

A	B	C
1	2	3
4	2	5

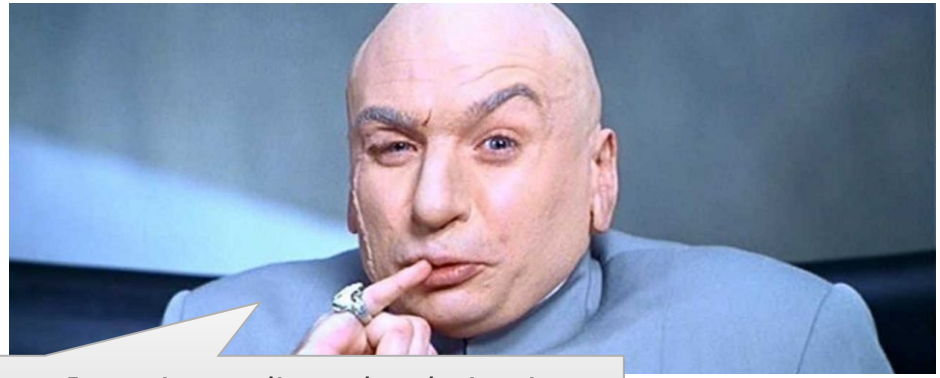
  - Projections on R_1(A,B) and R_2(B,C)

A	B
1	2
4	2

B	C
2	3
2	5

  - Restoring by Join over B provides an incorrect result:

A	B	C
1	2	3
1	2	5
4	2	3
4	2	5



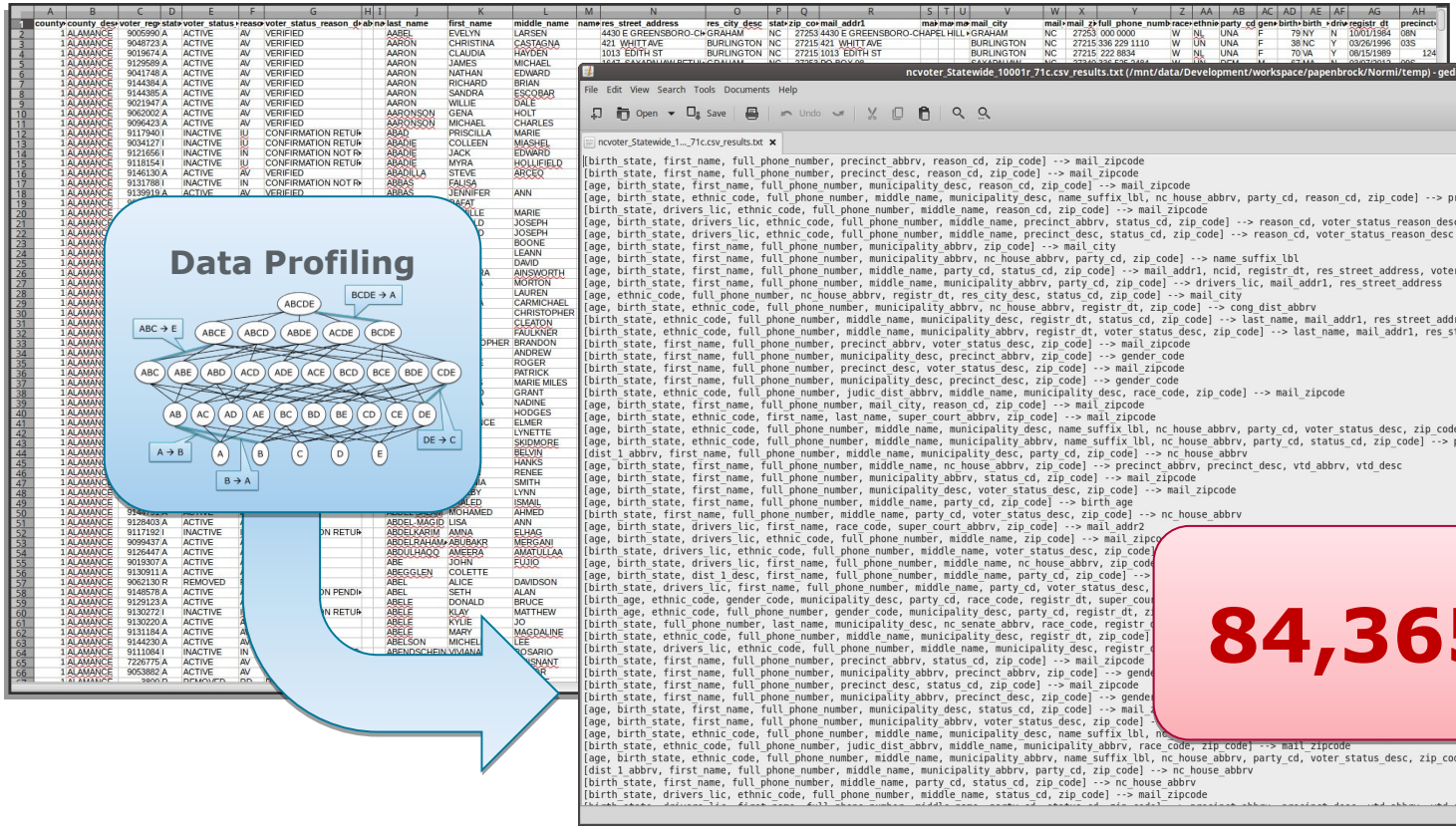
Then, I can just split each relation into two-attributed relations!

Only if it works with **lossless decomposition**!

Boyce-Codd Normal Form Automation

Data-driven Schema Normalization,
T. Papenbrock and F. Nauman, EDBT, 2017

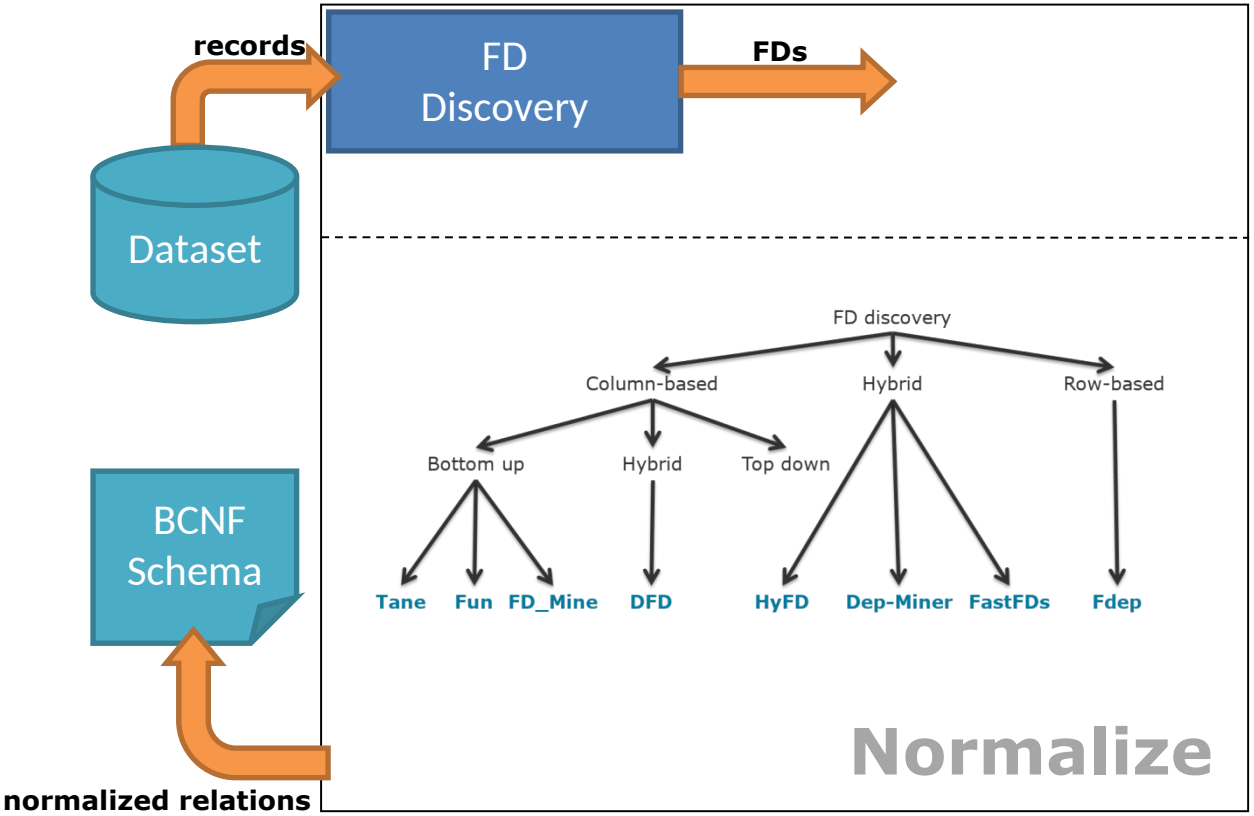
Philipps Universität Marburg

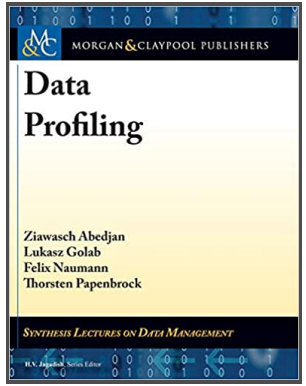


Boyce-Codd Normal Form Automation

Data-driven Schema Normalization,
T. Papenbrock and F. Nauman, EDBT, 2017

Philipps Universität Marburg



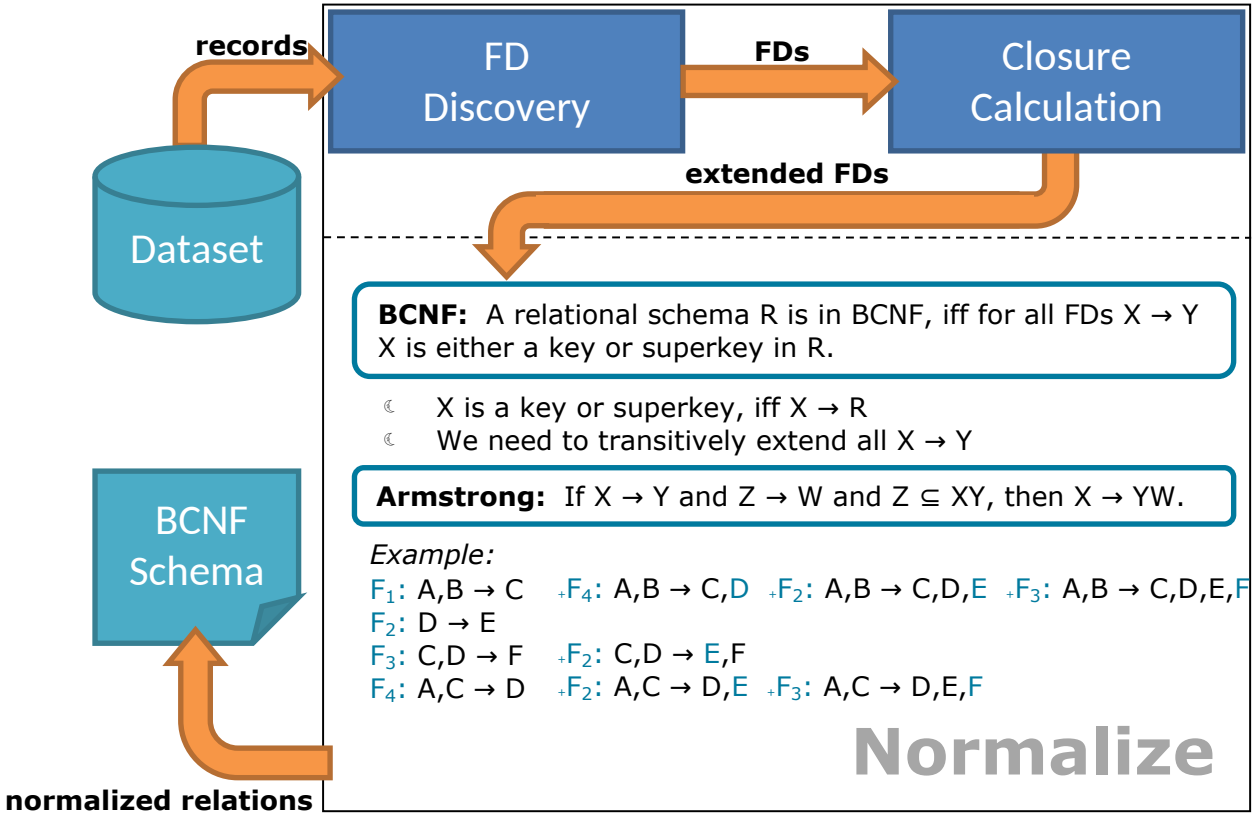


Database Systems
Normalization

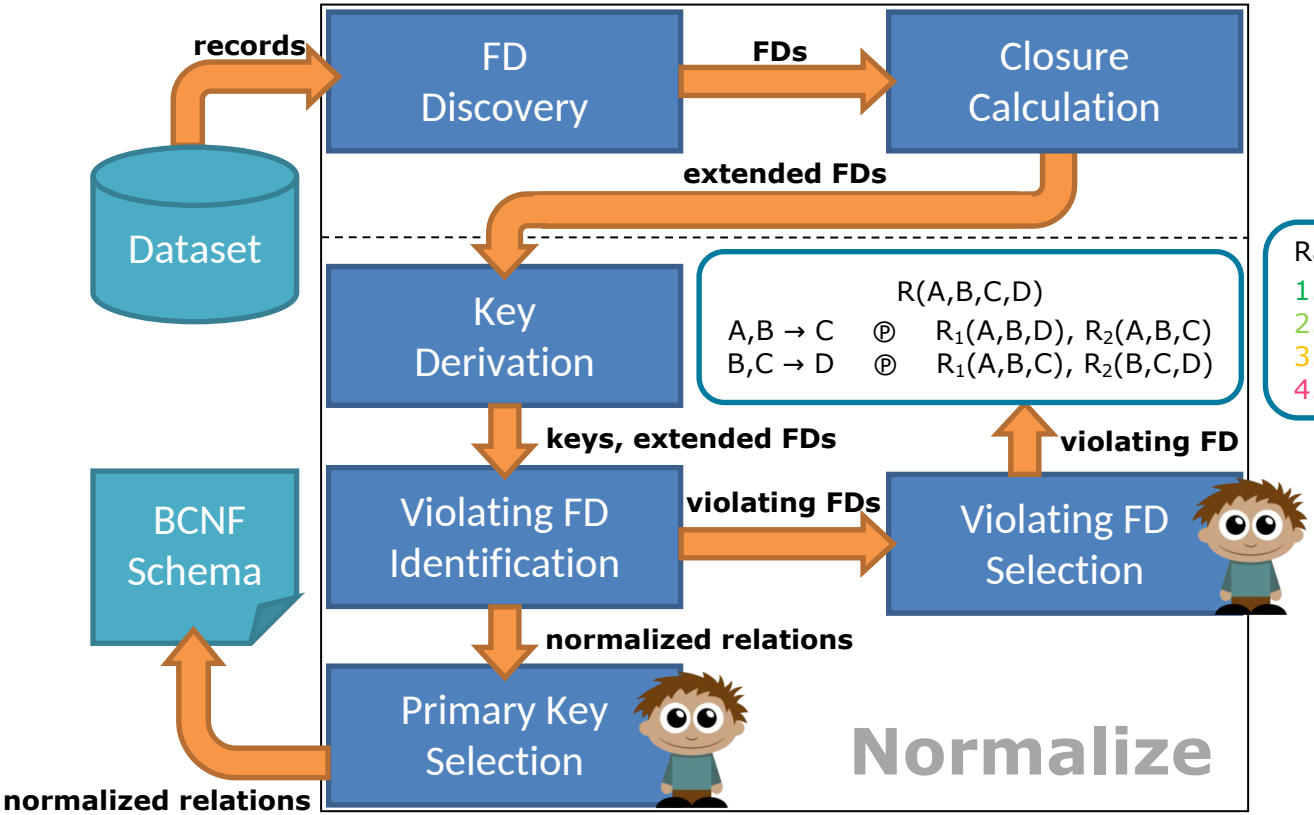
Boyce-Codd Normal Form Automation

Data-driven Schema Normalization,
T. Papenbrock and F. Nauman, EDBT, 2017

Philipps Universität Marburg



Boyce-Codd Normal Form Automation



Ranking by likelihood

1. A,B → C,D,E
2. B,C → D,E
3. A,C,D → E
4. C,D,E → A

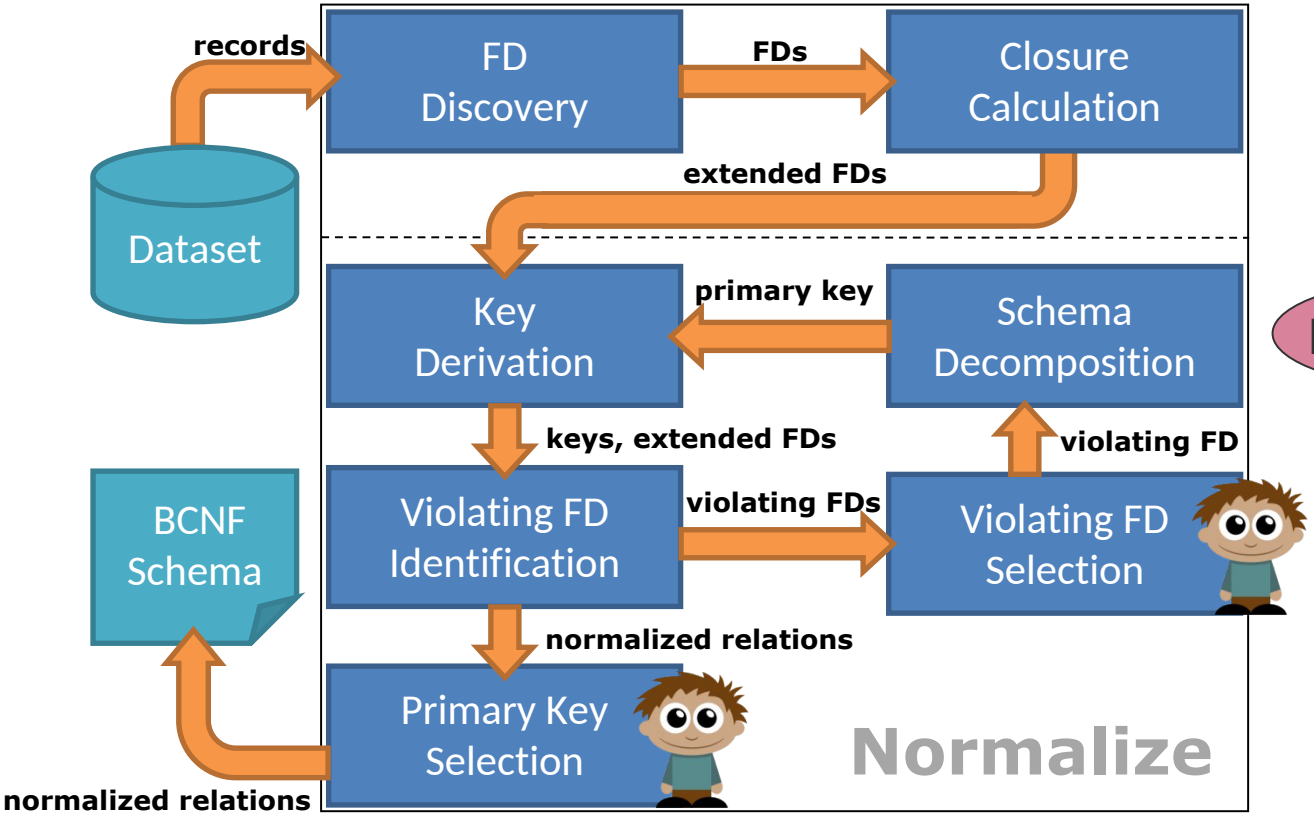
score(FD) :=
lengthScore(FD) +
valueScore(FD) +
positionScore(FD) +
duplicationScore(FD)

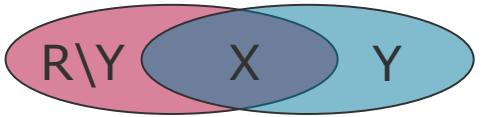
Normalization

Boyce-Codd Normal Form Automation

Data-driven Schema Normalization,
T. Papenbrock and F. Nauman, EDBT, 2017

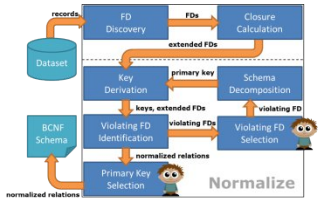
Philipps Universität Marburg





Database Systems
Normalization

Boyce-Codd Normal Form
Automation – Results



Philipps Universität Marburg

(**a_id**, **p_id**, **rl_id**, **t_id**)

- (c_id, **t_id**, t_gid, t_medium, t_number, t_name, t_length, t_lupdated) — **TRACK**
  - (**c_id**, t_ac, **t_lupdated**)
  - (**c_id**, t_position, **t_number**)
  - (**c_id**, c_gid, c_name, c_length, c_comment, c_lupdated) — **RECORDING**
    - (**c_name**, c_edits_pending)
- (**p_id**, p_gid, p_name, p_address, p_coordinates, p_comment, p_lupdated, p_byear, p_bmonth, p_bday, p_eyear, p_emonth, p_eday, p_ended) — **PLACE**
  - (**p_name**, p_type, p_edits_pending)
- (e_id, **a_id**, a_gid, a_name, a_sort_name, a_byear, a_bmonth, a_bday, a_eyear, a_emonth, a_eday, a_gender, a_comment, a_lupdated, acn_name) — **ARTIST**
  - (**a_lupdated**, a_ended, a_barea, a_earea, acn_position, acn_joinp) — **ARTIST_CREDIT_NAME**
  - (**e_id**, e_gid, e_name, e_type, e_lupdated) — **AREA**
- (l_id, r_id, **rl_id**, rl_cnumber, rl_lupdated) — **RELEASE_LABEL**
  - (**l_id**, l_gid, l_name, l_byear, l_bmonth, l_lcode, l_area, l_comment, l_lupdated, l_ended) — **LABEL**
    - (**l_name**, l_bday, l_emonth, l_eday, l_type)
    - (**l_name**, l_eyear, **l_ended**)
  - (**r_id**, r_gid, r_name, r_ac, r_status, r_packaging, r_language, r_script, r_barcode, r_comment, r_edits_pending, r_quality, r_lupdated, rg_id) — **RELEASE**
    - (ac_id, **rg_id**, rg_gid, rg_name, rg_type, rg_lupdated) — **RELEASE_GROUP**
      - (rg_comment, rg_editspending, **rg_lupdated**)
      - (e_byear, e_bmonth, e_bday, a_type, **ac_id**, ac_name, ac_acount, ac_refcount, ac_created) — **ARTIST_CREDIT**
        - (e_edits_pending, e_eyear, e_emonth, e_eday, e_ended, a_edits_pending, **ac_ref_count**, **ac_created**, l_editspending, c_video, t_editspending, t_isdtrack)

- (**linenumber**, extendedprice, discount, tax, returnflag, shipdate, commitdate, receiptdate, comment, **orderkey**, partkey) — **LINEITEM**
  - (**linenumber**, **extendedprice**, **tax**, **commitdate**, **receiptdate**, shipinstruct)
  - (**extendedprice**, **discount**, shipmode, **orderkey**)
  - (quantity, **extendedprice**, **partkey**)
  - (linestatus, **shipdate**)
  - (**tax**, **returnflag**, **orderkey**, **partkey**, suppkey)
    - (availqty, supplycost, comment, **partkey**, **suppkey**) — **PARTSUPP**
      - (**partkey**, name, brand, type, size, container, retailprice, comment) — **PART**
        - (mfgr, **brand**)
      - (**suppkey**, name, address, phone, acctbal, comment, nationkey) — **SUPPLIER**
        - (**nationkey**, name, comment, regionkey) — **NATION**
          - (shippriority, **regionkey**, name, comment) — **REGION**
- (**orderkey**, totalprice, orderdate, orderpriority, clerk, comment, custkey) — **ORDERS**
  - (orderstatus, **totalprice**, **orderdate**)
  - (**custkey**, name, address, phone, acctbal, mktsegment, comment) — **CUSTOMER**



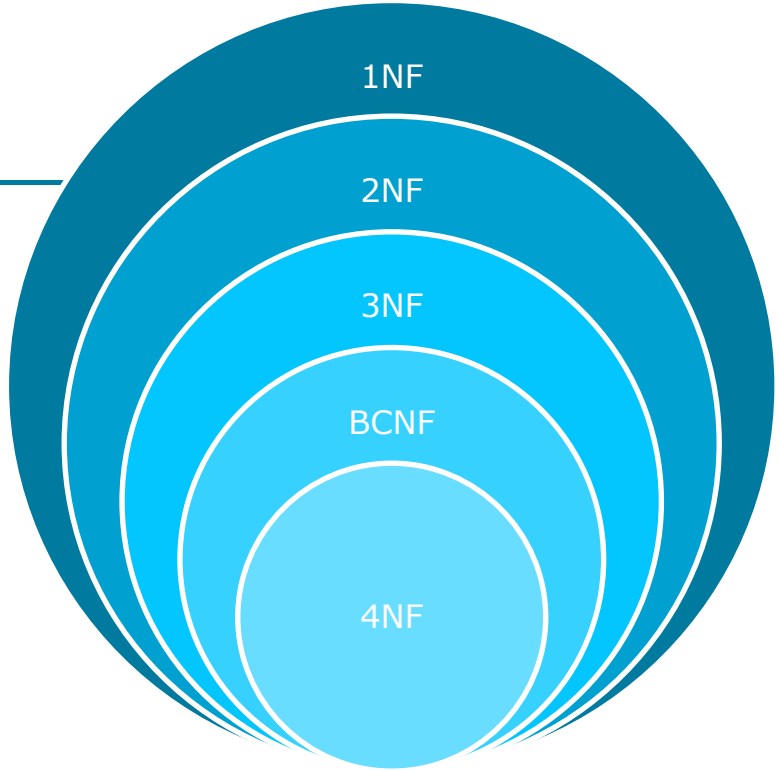
Normalization

- Functional Dependencies
- Closure Algorithm
- Boyce-Codd Normal Form
- **Normal Forms**



Normal Forms
Overview

- 1. Normal Form (1NF)
  - Only atomic values
- 2. Normal Form (2NF)
  - 1NF and no FD of a part of a key
- 3. Normal Form (3NF)
  - 2NF and no transitive FDs
- Boyce-Codd Normal Form (BCNF)
  - 3NF and no transitive FDs, also within the key
- 4. Normal Form (4NF)
  - BCNF and no multivalued dependencies (MvD)
- Essential tuple normal form (ETNF)
- Redundancy-free normal form (RFNF)
- Superkey normal form (SKNF)
- Projection-join normal form (5NF)



1. Normal Form (1NF)

- Definition
  - A relation schema is in the 1. Normal Form (1NF), if all attributes can accept only atomic values that are no more splittable.
    - Value ranges such as STRING, INTEGER, FLOAT, BOOLEAN etc.
    - Sets of values, complex objects or other structures (e.g. Tuples, Arrays, XML etc.) are not allowed as attribute values.

- Remarks
  - 1. Normal Form was already required a relation by definition.
  - Today's relational systems offer possibilities to generate relations in Non-First-Normal-Form (aka. NFNF, NF2 or NF^2).
    - Example: Array support in PostgreSQL

Normal Forms
1. Normal Form (1NF)

- 1NF Normalization
  - Option 1: Flatten
  - Option 2: Map

Father	Mother	Children
Johann	Martha	{Else, Lucie}
Johann	Maria	{Theo, Josef}
Heinz	Martha	{Cleo}

Violates 1NF

1

Father	Mother	Child
Johann	Martha	Else
Johann	Martha	Lucie
Johann	Maria	Theo
Johann	Maria	Josef
Heinz	Martha	Cleo

Key adjustment from {Father, Mother} to {Father, Mother, Child} is necessary

2

Father	Mother	Child1	Child2
Johann	Martha	Else	Lucie
Johann	Maria	Theo	Josef
Heinz	Martha	Cleo	NULL

Less flexibility and NULL values

Normal Forms
2. Normal Form (2NF)

Philipps Universität Marburg

- Definition
  - A relation schema is in the 2. Normal Form (2NF), if no non-key-attribute of only a part of a key is independent.
    - ➢ There is no nontrivial FD X → A for which X is a subset of a key.

- Example: ToR(<u>StNR, CourseID</u>, CourseTitle, RoomNR, Grade)
  - FA 1: StNR, CourseID → Grade
  - FA 2: CourseID → CourseTitle, RoomNR
  - FA 2 violates the 2NF which might lead to anomalies, e.g.:
    - Insert: Course information are only available, when the course has already one Transcript of Records (ToR) has generated.
    - Update: Course information are saved redundantly.
    - Delete: Course information are lost, if the last ToR is deleted.

2. Normal Form (2NF)

- 2NF Normalization
  - Decomposition of the 2NF violating schemas to violating FDs

- Example: ToR(StNR, CourseID, CourseTitle, RoomNR, Grade)
  - FA 1: StNR, CourseID → Grade
  - FA 2: CourseID → CourseTitle, RoomNR
  - Decomposition with FA 2:
    - ToR(StNR, CourseID, Grade)
    - Course(CourseID, CourseTitle, RoomNR)

#key attributes ≥ 2

- Remarks
  - 2NF can only be violated, if the keys are merged.
  - In practice, the meaning of the 2NF is minor, since redundancy is only partially triggered.

3. Normal Form (3NF)

- Definition
  - A relation schema is in the 3. Normal Form (3NF), if there is no non-key-attributes, that are transitively dependent on a key.
    - For each nontrivial FD $X \rightarrow A$, X is a (super)key or A is a part of a key.

- Remarks
  - 3NF eliminates dependencies of nonkey attributes (= transitive dependencies).
  - Each relation in 3NF is also in 2NF.

3. Normal Form (3NF)

- 3NF Decomposition Algorithm:
  - Determine the canonical coverage F_C from a given set F of FDs:
    - Left reduction for all $X \rightarrow Y \in F$: if $Y \subseteq \{X \backslash A\}^+$, then replace with $X \backslash A \rightarrow Y$.
    - Right reduction for all $X \rightarrow Y \in F$: if $B \subseteq \{X\}^+$ and $X \rightarrow Y \backslash B$, then replace with $X \rightarrow Y \backslash B$.
    - Clean all $X \rightarrow \{\} \in F$: Remove, if $Y = \{\}$.
    - Aggregation of all $X \rightarrow Y_1, \ldots, X \rightarrow Y_n \in F$: Combine all FDs with the same X to $X \rightarrow Y_1 \cup \ldots \cup Y_n$.
  - Create relation schemas $R_X := X \cup Y$ for all $X \rightarrow Y \in F_C$:
    - R_X contains all FDs $F_X = \{X' \rightarrow Y' \in F_C \mid X' \cup Y' \subseteq R_X\}$
  - Reconstruct keys/key candidates:
    - If key (candidate) from R was preserved with respect to F_C, it is retained.
    - Otherwise choose a key(candidate) $S \subseteq R$ and define the following additional schema: $R_S := S$ with $F_S := \{\}$
  - Eliminate redundant relations:
    - Removing of all schemas R_X, that are contained in others (i.e. $R_X \subseteq R_{X'}$)

Byce-Codd Normal Form (BCNF)

We have already thoroughly discussed

- Definition
  - A relation schema is in the Boyce-Codd Normal Form (BCNF), iff for each nontrivial FD $X \rightarrow A$ that attributes X are any keys.
    - ➢ The left-hand side of each valid, nontrivial FD contains a key.
- Recall
  - Comparing to 3NF, the BCNF decomposition dismantles the keys and FDs as well if necessary.
    - ➢ 3NF is closure-preserving, on the other hand BCNF not!
- Addition: Closure-Preserving
  - A dismantlement of R to R_1, …, R_n is closure-preserving iff the set of the FDs F of R is verifiable locally on the individual relations via their FDs F_1, …, F_n
    - Formal: $F^+ = (F_1 \cup \ldots \cup F_n)^+$

Normal Forms
Byce-Codd Normal Form (BCNF)

Philipps Universität Marburg

3NF vs. BCNF is a trade-off between Closure-Preserving and redundancy minimization

- Example: 3NF vs. BCNF
  - CinemaPerformances(Title, Cinema, City)
  - FDs Cinema → City (a cinema may be exist only in a city)
    Title, City → Cinema (a film is shown only once in the same city)
  - Keys?
    - {Title, City} because of Title, City → Cinema
    - {Cinema, Title} because of Cinema → City
  - 3NF violations: no since City is a part of a key
  - BCNF violations: Cinema → City since Cinema is not a super-key
  - BCNF decomposition:
    - R_1(Cinema, City)
    - R_2(Cinema, Title)
  - Problem: BCNF decomposition violates the Hüllentreue since the FD Title, City → Cinema cannot be ensured anymore!

R_1

Kino	Title
Ufa	Star Wars
Babylon	Star Wars

R_2

Kino	Stadt
Ufa	Berlin
Babylon	Berlin

$R_1 \bowtie R_2$

Kino	Stadt	Title
Ufa	Berlin	Star Wars
Babylon	Berlin	Star Wars

4. Normal Form (4NF)

- Definition
  - A relation schema is in the 4. Normal Form (4NF), if it is in BCNF and for each nontrivial multivalued dependency (MvD) X Y the attribute sets X are a key.

- Multivalued Dependency
  - A multivalued dependency (MvD) X Y in a relation schema R with a relational instance r and X ⊆ R and Y ⊆ R is a valid expression, iff
    $\forall t_i, t_j \in r : t_i[X] = t_j[X] \Rightarrow \exists t_k \in r : t_k[XY] = t_i[XY] \wedge t_k[R \backslash Y] = t_j[R \backslash Y]$.
    - „If X appears with two different Y values, then it also appears interchanged with the following R\Y values.“

a.k.a. „Join Dependencies“

4. Normal Form (4NF)

- Example: Multivalued Dependencies

	name	street	city	email	provider
t_1	M. Walker	42 Home St.	Springfield	m.walker@uni.edu	EDUCAUSE
t_2	M. Walker	73 Work Ln.	Franklin	m.walker@uni.edu	EDUCAUSE
t_3	M. Walker	42 Home St.	Springfield	mia87@gmail.com	Gmail
t_4	M. Walker	73 Work Ln.	Franklin	mia87@gmail.com	Gmail
t_5	M. Walker	42 Home St.	Springfield	miwa123@aol.de	AOL
t_6	M. Walker	73 Work Ln.	Franklin	miwa123@aol.de	AOL

name ↠ street, city name ↠ email, provider

- Decomposition:
  - Addresses(name, street, city)
  - Contacts(name, email, provider)

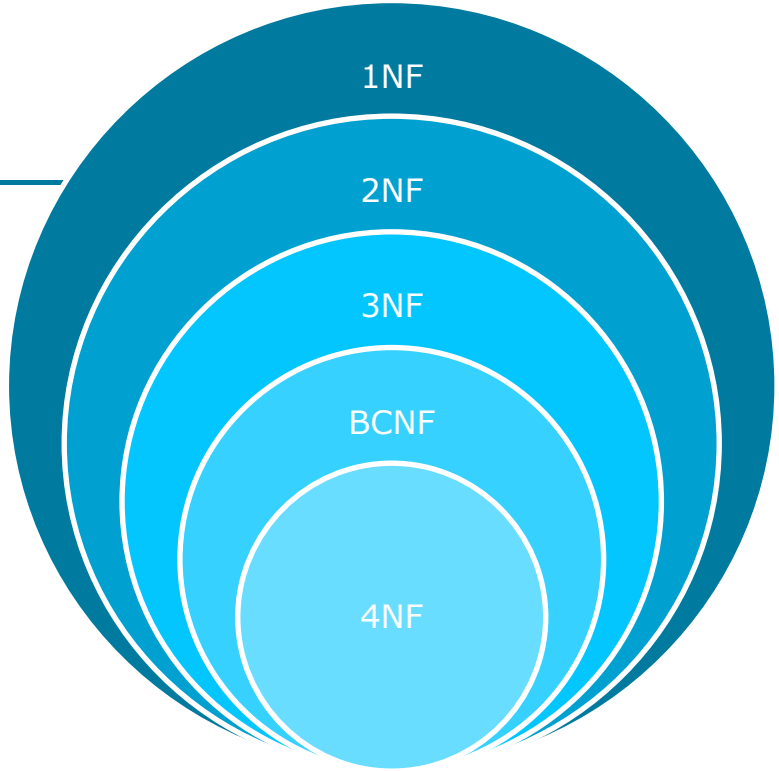
$t_1(N,S_1,C_1,E_1,P_1)$ $t_2(N,S_2,C_2,E_2,P_2)$



$t_3(N,S_1,C_1,E_2,P_2)$ & $t_4(N,S_2,C_2,E_1,P_1)$

Normal Forms
Overview

- 1. Normal Form (1NF)
  - Only atomic values
- 2. Normal Form (2NF)
  - 1NF and no FD of a part of a key
- 3. Normal Form (3NF)
  - 2NF and no transitive FDs
- Boyce-Codd Normal Form (BCNF)
  - 3NF and no transitive FDs, also within the key
- 4. Normal Form (4NF)
  - BCNF and no multivalued dependencies (MvD)
- Essential tuple normal form (ETNF)
- Redundancy-free normal form (RFNF)
- Superkey normal form (SKNF)
- Projection-join normal form (5NF)



Relevancy is mostly academic than practical; therefore not here.

Normalization

- Functional Dependencies
- Closure Algorithm
- Boyce-Codd Normal Form
- Normal Forms



Normalization

1. Requirement Analysis
   - Informal description
2. Conceptional Design
   - Entity-Relationship Diagram
3. Distribution Design
   - Partitioning
4. Logical Design
   - Relation schemas
5. Data Definition
   - SQL DDL
6. Physical Design
   - Parameters and Indices
7. Implementation and Maintenance
   - Installations, Backups, etc.



Summary

- Relational model
- Schemas
- From entity types to relations
- From relationship types to relations
- From ISA-hierarchies to relations
- Merging
- Functional Dependencies
- Key of a relation
- Rules for FDs
- Decomposition
- BCNF
- Normal Forms

